

AIX30006 Algorithm Project 1

Genetic Algorithm

Team 1

21700586 Jinhyun Lee

22000724 Eunbi Cho

April 09, 2023



AIX30006 Project – Genetic Algorithm

https://s3-us-west-2.amazonaws.com/secure.notion-static.com/e92616f4-cce2-4d45-a4a5-661eac31679a/Genetic_algorithm_project_description.pdf

Introduction

Genetic algorithms are a type of optimization algorithms inspired by the process of natural selection and genetics. They're based on the idea of evolution. Populations go through a process of reproduction and selection to produce a new generation of individuals that better adapt to their environment.

In the context of genetic algorithms, individuals in the population represent potential solutions to problems. Each individual is encoded into a set of genes, which can be thought of as a variable or parameter defining the solution. The algorithm then applies a series of operations such as selection, crossover, and mutation to create a new generation of individuals.

Selection is the process of selecting the most appropriate individual in the population to become the next generation of parents. Crossover involves exchanging genetic material between pairs of parents to create new offspring that combine their characteristics. Mutations involve randomly changing genes in an individual's genome to introduce new mutations.

The selection, crossover, and mutation processes are repeated over generations until a satisfactory solution is found. The goal is to maximize fitness, a measure of how well each individual performs in solving problems.

The great thing about Genetic Algorithm become obvious when compared to Gradient Descent algorithm. Gradient descent algorithm is a good algorithm to find a global minimum point by adjusting the position according to the step size, but depending on where you start the first position, you can reach the global minimum or local minimum. However, the great advantage of genetic algorithm is that it can reach the global minimum wherever it starts.

By optimizing a continuous function using the algorithm and solving the Traveling Salesman Problem we will Genetic Algorithm we will exercise the implementation of the genetic algorithm.

Task 1. Optimize a continuous function using the genetic algorithm

Given the objective function of:

$$f(x) = 2x^4 - 10x^3 + 8x^2 + 5x$$

asked to optimize the solution, using the genetic algorithm, under unconstrained and constrained environment using own genetic algorithm code.

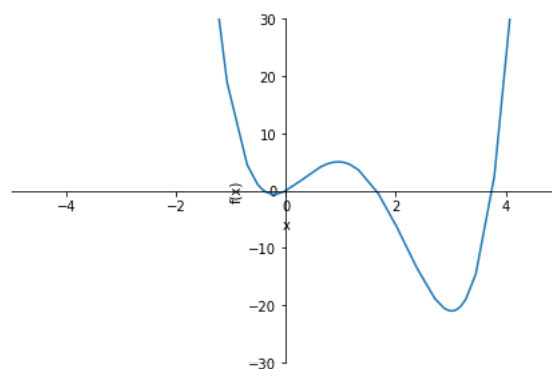


Figure 1: graph of objective function

As the graph shows, the most global minimum point is located at $x = 3.019$. However, not only $x = 3.019$, but there is also a point at $x = -0.218$ that slope is 0. Let's assume that you use gradient descent at this time, if the start point is $x = 2$ or $x = 4$, it may converge to $x = 3.019$, but if $x = -2$, it will eventually converge to $x = -0.218$ which is not global minimum point.

We will find out how to solve this problem through genetic algorithm:

hyperparameter setting

```
# ===== setting the hyperparameters(user-defined) =====

# setting the max iteration
max_iter = 200

# number of variable
var_num = 1

# number of bit (usually it's 16)
bits = 16

# bound of the function
bounds = (-5, 5)

# number of first generation
pop_num = 20

# number of next generation
next_generation = 20

# size of tournament selecting
tournament_size = 16

# rate of crossover is realized
cross_rate = 0.8

# rate of mutation is realized
muta_rate = 0.1

# setting objective function
x = symbols('x')
fx = 2 * x ** 4 - 10 * x ** 3 + 8 * x ** 2 + 5 * x
```

First of all, since the objective function is a two-dimensional function, there is only one variable. Therefore, `var_num` is set to 1. The interval of the function is set to -5, 5. In addition, **the first generation population was set to 20**, and the next generation was set to 20. The `Tournament_size` was set to 16, allowing 80% of the population (16/20) to participate in the tournament randomly. The `cross_rate`, which is the probability of crossover occurring, was first set to 0.8 which is commonly used, and the `muta_rate` was set to 0.1 for the same reason.

testing why 20 could have been better option

Unconstrained optimization

Overall code for implementing genetic algorithm.

```
def genetic_algorithm():

    iteration = 0
    bit_list = bit_string(var_num, bits, pop_num) ## making the bitstring as number of populations
    prev_value = 0

    fit_list = []

    while iteration < max_iter:

        decoding = decode(bit_list, bits, bounds) ## decoding value (x - value)

        pop_df, fit_value = pop_val(decoding, fx) # y-value

        fit_list.append(fit_value)
        ## evaluate fitness function values
        if prev_value == fit_value: ## Terminate if the previous value and the current value are the same
            break

        prev_value = fit_value # update the prev_value

        t_df = tournament_selection(pop_df, tournament_size) # tournament selection

        new_list = []

        while len(new_list) < next_generation: ## Repeat until the next generation is number 'next_generation'

            new_list.extend([crossover(t_df, 0.8)[0], crossover(t_df, 0.8)[1]])
            new_list.append(mutation(t_df, bits, 0.2))

            if len(new_list) > next_generation:
                del new_list[-1]
```

```

decoding_new = decode(new_list, bits, bounds)

new_df, _ = pop_val(decoding_new, fx)

all_df = pd.concat([pop_df, new_df], axis=0) ## Concatenate dataframe of the present generation and the next generation

# Select only the top 50%
all_df = all_df.sort_values(by = 'y', ascending = True)
select_df = all_df[: int(len(all_df)/2)]

bit_list = list(select_df.bitstring)

iteration += 1

print('number of iterations: {}'.format(iteration))

return pop_df

```

First of all, functions such as `decode` and `pop_val`, `tournament_selection`, `crossover`, and `mutation` are user-defined functions, and the code for this is detailed in the code simulation file. `genetic_algorithm` is created by getting all pre-defined functions above. The function is iterated by `max_iter` (set to 200) through the while statement, and if the current `fit_value` value is equal to the previous `fit_value` value, the while statement is interrupted. The `fit_value` refers to the average of the function values(value of y) of the generation genes.

Then, 16(`tournament_size`) are randomly extracted from the current generation's genes through tournament selection, and a 1:1 match is made. Since the problem is to find the minimum value of the objective function, the smaller the fitness function value, the more dominant the gene is. Therefore, a gene with a smaller y-value wins. The eight individuals who won the 1:1 match are crossed over and mutated, and they are reproduced until the next generation population reaches 20.

Then, collect the parents and children, and then sort the genes in order of the lowest y values, and send only 50% to the next generation. Repeat this process.

explanation

The following is a result of executing `genetic_algorithm` and outputting `pop_df`(Pandas dataframe). The Iteration has been executed 20 times.

	bitstring	x	y
0	1100110100110001	3.015412	-21.0092
1	1100110100110001	3.015412	-21.0092
2	1100110110000001	3.027619	-21.0075
3	1100110011110001	3.005646	-21.0048
4	1100110011110001	3.005646	-21.0048
5	1100110011110001	3.005646	-21.0048
6	1100110011110001	3.005646	-21.0048
7	1100110011110001	3.005646	-21.0048
8	1100110011110001	3.005646	-21.0048
9	1100110011110001	3.005646	-21.0048
10	1100110011110001	3.005646	-21.0048
11	1100110011110001	3.005646	-21.0048
12	1100110011110001	3.005646	-21.0048
13	1100110011110001	3.005646	-21.0048
14	1100110011110001	3.005646	-21.0048
15	1100110011110001	3.005646	-21.0048
16	1100110011110001	3.005646	-21.0048
17	1100110011110001	3.005646	-21.0048
18	1100110011110001	3.005646	-21.0048
19	1100110011110001	3.005646	-21.0048

Table 1: result of executing the `genetic_algorithm`(function)

Bitstring values and x and y values of 20 genes are stored in the data frame. These genes are converging on certain points. The minimum value of the function was in fact x = 3.019, indicating that it did not converge to the exact minimum value, but that

it converged to a value which is similar with minimum value, indicating that genetic-algorithm is a meta-hueristic algorithm. In other words, since it takes a long time to find the exact value, it can be said that genetic algorithm provides a "near" good solution to the optimal solution in a short time.

Constrained optimization

Then, suppose that the objective function is subject to the constraint:

$$x \leq 2$$

This means that x can no longer go beyond 2.

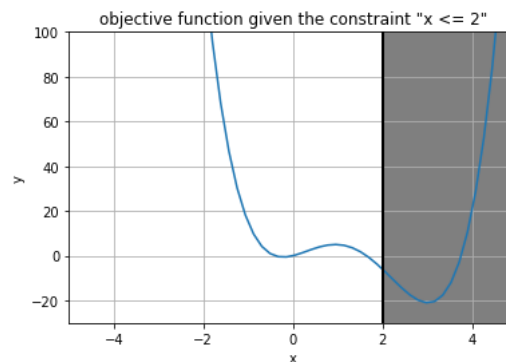


Figure 2: graph of objective function with constraint $x \leq 2$

Since there is no way to handle construction in Genetic Algorithm, a penalty function must be specified to prevent x from exceeding 2. Penalty function makes the y - value very large when x goes over 2 so that x does not go further than 2. So, as you can see from the graph, $x = 1.99999...$

($y = -5.99999...$) is could be the best point under the constraint.

```
# ===== hyperparameter for constraint =====

penalty_factor = 100000
epsilon = 0.1 # this is for preventing the value to cross the certain value under the constraint.
```

Defining the **penalty function**:

```
def pop_val_penalty(decoding, fx, penalty_factor, epsilon):

    pop_val = []
    pop_df = decoding

    for i in range(len(decoding)):

        ## constraints x <= 2
        penalty_function = 0

        if pop_df['x'][i] > 2:

            ## Add epsilon to give a penalty that is even very close to 2 such as 2.000076.
            penalty_function = penalty_factor * (max(0, pop_df['x'][i] - (2 - epsilon)))**2
            pop_val.append(fx.subs([(x, pop_df['x'][i])]) + penalty_function)

        else:
            pop_val.append(fx.subs([(x, pop_df['x'][i])]))

    pop_df['y'] = pop_val

    pop_df = pop_df.astype({'x': 'float',
                           'y': 'float'})

    fit_value = np.mean(pop_df.y)
```

```
return pop_df, fit_value
```

In this function, if x exceeds 2, the value of x is added to the penalty function to add the `penalty_function` to the existing value. At this time, the larger the `panalty_factor`, the greater the penalty. As you may guess, closer to 2, smaller the penalty. So, reason why `epsilon` is added here is to give a large penalty even for small values such as 2.000076.

Overall code for implementing genetic algorithm with constraint.

```
def genetic_algorithm_constraint():

    iteration = 0
    bit_list = bit_string(var_num, bits, pop_num) ## making the bitstring as number of populations
    prev_value = 0

    fit_list = []

    while iteration < max_iter:

        decoding = decode(bit_list, bits, bounds) ## decoding value

        # if x >= 2, get penalty
        pop_df, fit_value = pop_val_penalty(decoding, fx, penalty_factor, epsilon)

        fit_list.append(fit_value)
        ## evaluate fitness function values
        if prev_value == fit_value:
            break

        prev_value = fit_value # update

        t_df = tournament_selection(pop_df, tournament_size) # tournament

        new_list = []

        while len(new_list) < next_generation: ## Repeat until the next generation is number 'next_generation'

            new_list.extend([crossover(t_df, 0.8)[0], crossover(t_df, 0.8)[1]])
            new_list.append(mutation(t_df, bits, 0.2))

            if len(new_list) > next_generation:
                del new_list[-1]

        decoding_new = decode(new_list, bits, bounds)

        new_df, _ = pop_val(decoding_new, fx)

        all_df = pd.concat([pop_df, new_df], axis=0)

        # selection
        all_df = all_df.sort_values(by = 'y', ascending = True) ## Concatenate dataframe of the present generation and the next genera
        select_df = all_df[: int(len(all_df)/2)]

        bit_list = list(select_df.bitstring)

        iteration += 1

    print("number of iterations: {}".format(iteration))

    return pop_df
```

Why epsilon is needed?

So, if there is no epsilon, a situation occurs in which the value of the converged x exceeds 2. The following is the result of the average of x values when the `genetic_algorithm_constraint` function is executed 10 times.

```
## epsilon = 0

number of iterations: 15
index: 0, average of x: 1.796063172350653

number of iterations: 32
index: 1, average of x: 1.9921416037232014

number of iterations: 22
index: 2, average of x: 1.9994659342336156
```

```

number of iterations: 43
index: 3, average of x: 2.000228853284504

number of iterations: 62
index: 4, average of x: 2.0000762951094835

number of iterations: 98
index: 5, average of x: 1.9994659342336156

number of iterations: 94
index: 6, average of x: 1.9994659342336156

number of iterations: 83
index: 7, average of x: 2.000228853284504

number of iterations: 125
index: 8, average of x: 2.0000762951094835

number of iterations: 13
index: 9, average of x: -0.21141374837872906

```

As you can see, there are quite a few cases that go over 2. So let's give epsilon 0.1 and then execute 10 times again.

```

## epsilon = 0.1

number of iterations: 17
index: 0, average of x: -0.2133974212252996

number of iterations: 31
index: 1, average of x: 1.8737315938048376

number of iterations: 14
index: 2, average of x: 1.989547570000763

number of iterations: 22
index: 3, average of x: 1.9968719005111772

number of iterations: 9
index: 4, average of x: 1.7142748149843587

number of iterations: 126
index: 5, average of x: 1.9921416037232014

number of iterations: 41
index: 6, average of x: 1.9993133440146487

number of iterations: 10
index: 7, average of x: -0.23323414969100487

number of iterations: 39
index: 8, average of x: 1.9994659342336156

number of iterations: 24
index: 9, average of x: 1.9884794384679942

```

If you give 0.1 to Epsilon, you can see that the value above 2 does not appear. Therefore, by adding epsilon, a large penalty is given to the value slightly over 2.

However, in the case of constrain, it was confirmed that it fell into the local minimum as $x = -0.23$ to -0.21 . Therefore, the solution for this situation is to find the optimal point by comparing the values with each other through several experiments (like executing 30 times) rather than conducting an experiment only once.

Next dataframe is the result of some execution of 'genetic_algorithm_constraint'

	bitstring	x	y
0	1011001100011110	1.996872	-5.94061
1	1011001011111111	1.992142	-5.85094
2	1011001011111110	1.991989	-5.84805
3	1011001011111110	1.991989	-5.84805
4	1011001011111110	1.991989	-5.84805
5	1011001011111110	1.991989	-5.84805
6	1011001011111110	1.991989	-5.84805
7	1011001011111110	1.991989	-5.84805

	bitstring	x	y
8	1011001011111110	1.991989	-5.84805
9	1011001011111110	1.991989	-5.84805
10	1011001011111110	1.991989	-5.84805
11	1011001011111110	1.991989	-5.84805
12	1011001011111110	1.991989	-5.84805
13	1011001011111110	1.991989	-5.84805
14	1011001011111110	1.991989	-5.84805
15	1011001011111110	1.991989	-5.84805
16	1011001011111110	1.991989	-5.84805
17	1011001011111110	1.991989	-5.84805
18	1011001011111110	1.991989	-5.84805
19	1011001011111110	1.991989	-5.84805

Table 2: Result of executing the genetic_algorithm_constraint(function)

In the unconstraint situation, it can be seen that the value converges near $x = 3.019$, while when constraint is given, it converges near $x = 1.999$. This shows that the optimal points are different from each other because the lowest point has to be found under the condition that x does not exceed 2.

Task 2. Address the Traveling Salesman Problem (TSP) using the genetic algorithm

Problem Statement

An operations research analyst is currently working for the United Parcel Service (UPS). The job is to provide the shortest route, which visits each delivery point exactly once from the origin point, to the UPS truck drivers. Each driver is responsible for one city and it is asked to manage seven drivers for planning seven different cities (i.e., Atlanta, Boston, Cincinnati, Denver, New York, Philadelphia, and San Francisco).

Process

1. Data Description:

There are seven different cities to manage: **Atlanta, Boston, Cincinnati, Denver, New York, Philadelphia, and San Francisco.**

Each city is divided into one dataframe in the format of Index/ Latitude/ Longitude.

Below is the representation of an Atlanta dataframe:

```
Atlanta.head()
```

	Index	Latitude (deg)	Longitude (deg)
0	0	33.665568	-84.411070
1	1	33.764940	-84.371819
2	2	33.770889	-84.358622
3	3	34.233391	-84.103278
4	4	33.921088	-84.337342

2. A plot showing the delivery locations for the selected cities (Atlanta, Philadelphia, Cincinnati).

```
plt.figure(figsize = (7, 8))

# plot showing the delivery locations for the selected cities
plt.scatter(atlanta['Longitude (deg)'], atlanta['Latitude (deg)'])

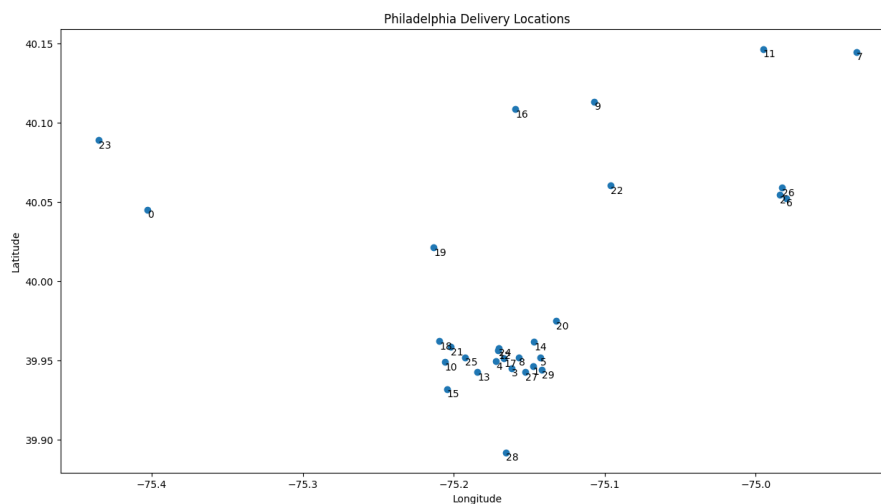
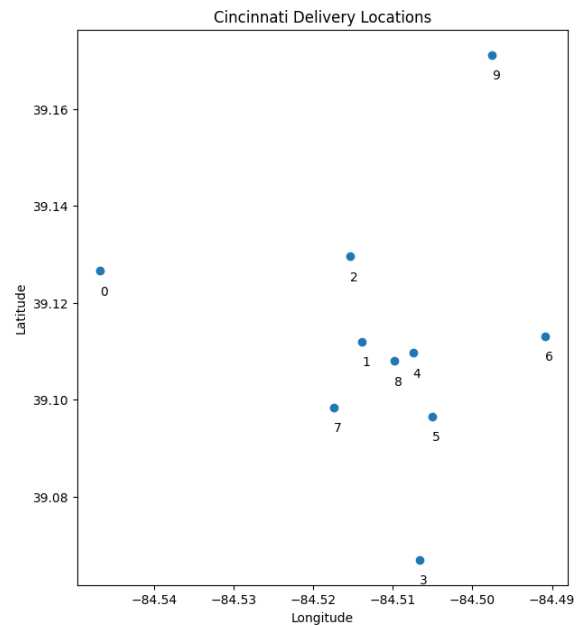
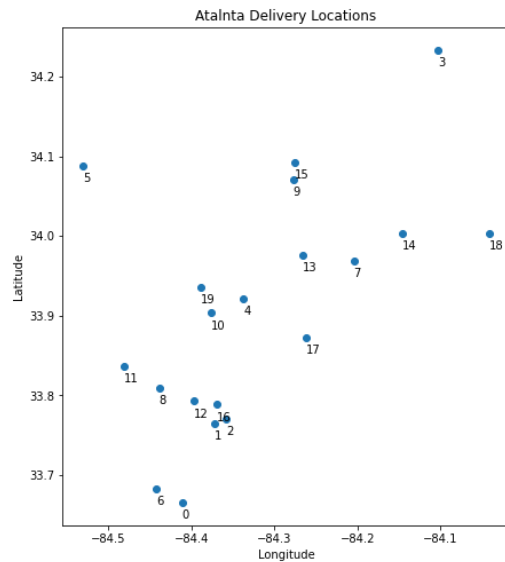
# Add labels and a title to the plot
```



```
plt.xlabel('Longitude')
plt.ylabel('Latitude')
plt.title('Atalanta Delivery Locations')

# Add location index as annotations to the plot
for i in range(len(atalanta)):
    plt.text(atalanta.loc[i, 'Longitude (deg)'], atalanta.loc[i, 'Latitude (deg)']-0.02, str(atalanta.loc[i, 'Index']))

# Show the plot
plt.show()
```



3. A distance adjacency matrix that represents great circle distances between the locations.

Here the Haversine formula was used to compute the distances.

Haversine Formula

The weights between two nodes is computed using haversine weight.

This formula is used to compute distance between two latitude and longitude coordinates with km as an unit:

$$d = r_{archav}(h) = 2r_{arcsin}(sq.root(h))$$

variable explanation

```
! pip install haversine
from haversine import haversine

for i in range(n):
    for j in range(i+1, n):
        coord_i = (atlanta.loc[i, 'Latitude (deg)'], atlanta.loc[i, 'Longitude (deg)'])
        coord_j = (atlanta.loc[j, 'Latitude (deg)'], atlanta.loc[j, 'Longitude (deg)'])
        dist = haversine(coord_i, coord_j, unit = 'm')
        d_a_matrix[i][j] = d_a_matrix[j][i] = dist
```

This returned the matrix of 20x20, that distance A_{ij} , represents the distance between the two delivery locations in a city.

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	0.000000	11630.796022	12676.072560	69228.326441	29218.010056	48212.458432	3485.530385	38713.657504	16259.198432	46724.087884	26629.830813	20102.841386	14219.741513	37016.491294	44783.748048	49027.666443
1	11630.796022	0.000000	1387.690177	57672.741630	17652.437139	38758.991621	11195.669589	27403.334147	7967.806360	35113.483706	15399.732942	12875.310669	3867.410333	25409.947854	33675.867547	37424.989254
2	12676.072560	1387.690177	0.000000	56558.610193	16816.616872	38631.286744	12458.764349	26177.427816	8597.067410	34186.222636	14827.682934	13482.389686	4287.644310	24344.952528	32406.556063	36508.975056
3	69228.326441	57672.741630	56558.610193	0.000000	40873.680964	42531.388324	68717.573557	30895.368938	56336.383181	24163.772666	44519.523208	56198.021370	55952.682119	32312.350466	25937.833434	22355.612738
4	29218.010056	17652.437139	16816.616872	40873.680964	0.000000	25699.527986	28186.051867	13340.286750	15513.641923	17543.950540	4150.190479	16258.764921	15273.023318	8984.434252	19845.987303	19826.381492
5	48212.458432	38758.991621	38631.286744	42531.388324	25699.527986	0.000000	45717.093977	32847.493247	32016.014716	23376.101282	24912.892675	28272.885094	35025.233887	27407.499481	36668.357376	23454.928635
6	3485.530385	11195.669589	12458.764349	68717.573557	28186.051867	45717.093977	0.000000	38596.768491	14109.928468	45724.134073	25228.664929	17455.457368	12917.137881	36407.007969	44863.278464	47989.439799
7	38713.657504	27403.334147	26177.427816	30895.368938	13340.286750	32847.493247	38596.768491	0.000000	27910.228925	13233.957424	17478.297396	29441.104591	26384.493237	5695.636515	6604.322634	15232.056913
8	16259.198432	7967.806360	8597.067410	56336.383181	15513.641923	32016.014716	14109.928468	27910.228925	0.000000	32608.228450	11860.553054	4908.642655	4334.990733	24408.394784	34503.070734	34775.487284
9	46724.087884	35113.483706	34186.222636	24163.772666	17543.950540	23376.101282	45724.134073	13233.957424	32608.228450	0.000000	20756.238168	32102.711737	32807.312498	10636.521414	14293.506915	2349.624255
10	26629.830813	15399.732942	14827.682934	44519.523208	4150.190479	24912.892675	25228.664929	17478.297396	11860.553054	20756.238168	0.000000	12148.573727	12419.797704	13046.741898	23995.631575	22944.422398
11	20102.841386	12875.310669	13482.389686	56198.021370	16258.764921	28272.885094	17455.457368	29441.104591	4908.642655	32102.711737	12148.573727	0.000000	9198.179920	25195.021087	36019.902575	34114.715647
12	14219.741513	3867.410333	4287.644310	55952.682119	15273.023318	35025.233887	12917.137881	26384.493237	4334.990733	32807.312498	12419.797704	9198.179920	0.000000	23662.788854	32866.702709	35072.784413
13	37016.491294	25409.947854	24344.952528	32312.350466	8984.434252	27407.499481	36407.007969	5695.636515	24408.394784	10636.521414	13046.741898	25195.021087	23662.788854	0.000000	11427.996629	12958.830418
14	44783.748048	33675.867547	32406.556063	25937.833434	19845.987303	36668.357376	44863.278464	6604.322634	34503.070734	14293.506915	23995.631575	36019.902575	32866.702709	11427.996629	0.000000	15544.174624
15	49027.666443	37424.989254	36508.975056	22355.612738	19826.381492	23454.928635	47989.439799	15232.056913	34775.487284	2349.624255	22944.422398	34114.715647	35072.784413	12958.830418	15544.174624	0.000000
16	14206.841197	2631.518711	2192.214690	55210.368669	15033.769310	36441.170054	13529.643418	25136.965430	6862.940988	32518.122701	12791.241594	11645.600779	2584.043996	22914.058677	31501.906756	34824.411093
17	26801.511230	15673.075185	14386.755534	42738.082949	8856.753411	34488.016866	26845.516476	11941.931825	17771.451545	22151.130894	11193.142920	20650.532937	15273.640739	11529.092549	18026.239870	24482.559053
18	50780.227063	40437.283448	39083.930312	26231.791673	28825.728007	46090.719868	51395.710733	15570.684642	42556.637476	23077.467542	32923.128750	44646.150744	40317.760408	20943.939707	9694.463250	23805.069353
19	30141.987808	19088.573272	18573.747321	42255.305931	5042.380507	21326.204293	28553.704980	17392.845243	14757.811810	18172.187417	3796.156610	13943.667843	15931.081302	12202.782947	23599.561535	20227.628659

20 rows x 20 columns (15 col. due to space)

Greedy Algorithm Implementation

“Greedy” means that it is designed to find a solution based on “nearest neighbor selection” from a starting point.

```
# Find a solution using the greedy algorithm:
def tsp_greedy(distances):
    num_cities = distances.shape[0]
    path = []
    visited = np.zeros(num_cities, dtype=bool)
    visited[0] = True

    while len(path) < num_cities:
        current_city = path[-1]
        nearest_city = np.argmin(distances[current_city][~visited])
        nearest_city = np.arange(num_cities)[~visited][nearest_city]
        path.append(nearest_city)
        visited[nearest_city] = True

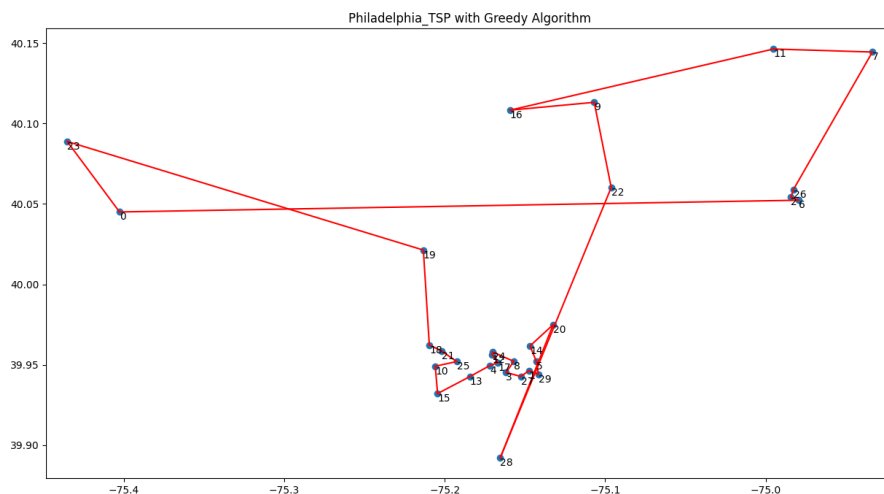
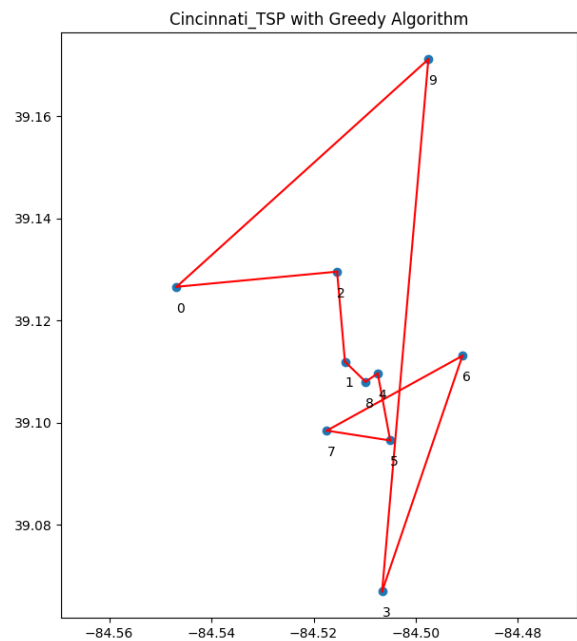
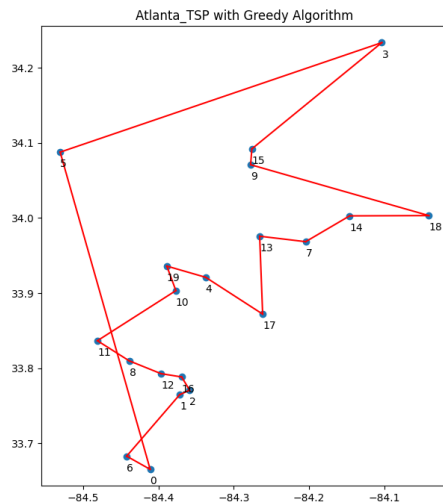
    return path + [0]
```

In the case of Atlanta, the algorithm returned the optimal solution of: **0, 6, 1, 2, 16, 12, 8, 11, 10, 19, 4, 17, 13, 7, 14, 18, 9, 15, 3, 5, 0.**

Visualizing the optimized route is represented in the graph below:

```
# Plot the cities and optimal path
plt.figure(figsize=(7, 8))
plt.scatter(atlanta['Longitude (deg)'], atlanta['Latitude (deg)'])
for i in range(len(atlanta)):
    plt.text(atlanta.loc[i, 'Longitude (deg)'], atlanta.loc[i, 'Latitude (deg)']-0.02, str(atlanta.loc[i, 'Index']))
for i in range(len(optimal_path)-1):
    start = atlanta.loc[optimal_path[i], ['Longitude (deg)', 'Latitude (deg)']]
    end = atlanta.loc[optimal_path[i+1], ['Longitude (deg)', 'Latitude (deg)']]
    plt.plot([start[0], end[0]], [start[1], end[1]], color='red')

plt.title('TSP with Greedy Algorithm')
plt.axis('equal')
plt.show()
```



The solution above returned the total distance of

- Atlanta: 231,983 meters
- Philadelphia: 157,122 meters
- Cincinnati: 34,353 meters

Given that a near-optimal solution for the Atlanta city is approximately 180,000 meters, by implementing a Genetic Algorithm we wanted to reach near the optimum solution.

Genetic Algorithm Implementation

The process implementing the genetic algorithm is as followed:

1. Install needed libraries

```
import numpy as np, random, operator, pandas as pd, matplotlib.pyplot as plt
```

2. Generate fitness function using great circle distance [haversine library](#)

```

#Handle city delivery locations and distance measure with Harversine library
!pip install haversine
from haversine import haversine, Unit
class City:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def distance(self, city):
        a=(self.x,self.y)
        b=(city.x,city.y)
        distance=haversine(a,b,unit=Unit.METERS)
        return distance

    def __repr__(self):
        return "(" + str(self.x) + "," + str(self.y) + ")"

#Fitness function (larger the better)
class Fitness:
    def __init__(self, route):
        self.route = route
        self.distance = 0
        self.fitness= 0.0

    def routeDistance(self):
        if self.distance ==0:
            pathDistance = 0
            for i in range(0, len(self.route)):
                fromCity = self.route[i]
                toCity = None
                if i + 1 < len(self.route):
                    toCity = self.route[i + 1]
                else:
                    toCity = self.route[0]
                pathDistance += fromCity.distance(toCity)
            self.distance = pathDistance
        return self.distance

    def routeFitness(self):
        if self.fitness == 0:
            self.fitness = 1 / float(self.routeDistance())
        return self.fitness

```

Created a `City` class that will allow us to create and handle our locations and we added a `distance` function using Haversine formula using python embedded library. Also created a `Fitness` class to treat the fitness as the inverse of the route distance. We want to minimize route distance, so a larger fitness score is better.

3. Create a random population:

```

def createRoute(cityList):
    route = random.sample(cityList, len(cityList))
    return route

def initialPopulation(popSize, cityList):
    population = []

    for i in range(0, popSize):
        population.append(createRoute(cityList))
    return population

```

4. Determine fitness:

```

def rankRoutes(population):
    fitnessResults = {}
    for i in range(0, len(population)):
        fitnessResults[i] = Fitness(population[i]).routeFitness()
    return sorted(fitnessResults.items(), key = operator.itemgetter(1), reverse = True)

```

Here we ranked each individual in the population. The output is an ordered list with the route IDs and each associated fitness score.

5. Selection:

```

def selection(popRanked, eliteSize):
    selectionResults = []

```

```

df = pd.DataFrame(np.array(popRanked), columns=["Index", "Fitness"])
df['cum_sum'] = df.Fitness.cumsum()
df['cum_perc'] = 100*df.cum_sum/df.Fitness.sum()

for i in range(0, eliteSize):
    selectionResults.append(popRanked[i][0])
for i in range(0, len(popRanked) - eliteSize):
    pick = 100*random.random()
    for i in range(0, len(popRanked)):
        if pick <= df.iat[i,3]:
            selectionResults.append(popRanked[i][0])
            break
    return selectionResults

def matingPool(population, selectionResults):
    matingpool = []
    for i in range(0, len(selectionResults)):
        index = selectionResults[i]
        matingpool.append(population[index])
    return matingpool

```

There are a few options for how to select the parents that will be used to create the next generation. We used **Roulette wheel selection** (also known as Fitness proportionate selection), which the fitness of each individual relative to the population is used to assign a probability of selection.

Also we used **elitism**, where best performing individuals from the population will automatically carry over to the next generation, ensuring that the most successful individuals persist.

6. Crossover:

```

#Ordered-Crossover
def breed(parent1, parent2):
    child = []
    childP1 = []
    childP2 = []

    geneA = int(random.random() * len(parent1))
    geneB = int(random.random() * len(parent1))

    startGene = min(geneA, geneB)
    endGene = max(geneA, geneB)

    for i in range(startGene, endGene):
        childP1.append(parent1[i])

    childP2 = [item for item in parent2 if item not in childP1]

    child = childP1 + childP2
    return child

def breedPopulation(matingpool, eliteSize):
    children = []
    length = len(matingpool) - eliteSize
    pool = random.sample(matingpool, len(matingpool))

    for i in range(0, eliteSize):
        children.append(matingpool[i])

    for i in range(0, length):
        child = breed(pool[i], pool[len(matingpool)-i-1])
        children.append(child)
    return children

```

The TSP is unique in that all locations must be included exactly one time. To abide by this rule, we used **ordered crossover**, where we randomly select a subset of the first parent string and then fill the remainder of the route with the genes from the second parent in the order in which they appear, without duplicating any genes in the selected subset from the first parent.

7. Mutation

```

#Mutation (Swap mutation)
def mutate(individual, mutationRate):
    for swapped in range(len(individual)):
        if(random.random() < mutationRate):
            swapWith = int(random.random() * len(individual))

            city1 = individual[swapped]
            city2 = individual[swapWith]

```

```

        individual[swapped] = city2
        individual[swapWith] = city1
    return individual

#Mutation through new population
def mutatePopulation(population, mutationRate):
    mutatedPop = []

    for ind in range(0, len(population)):
        mutatedInd = mutate(population[ind], mutationRate)
        mutatedPop.append(mutatedInd)
    return mutatedPop

```

Here we used `swap mutation` meaning that, with specified low probability, two cities will swap places in the route.

8. Repetition

```

def nextGeneration(currentGen, eliteSize, mutationRate):
    popRanked = rankRoutes(currentGen)
    selectionResults = selection(popRanked, eliteSize)
    matingpool = matingPool(currentGen, selectionResults)
    children = breedPopulation(matingpool, eliteSize)
    nextGeneration = mutatePopulation(children, mutationRate)
    return nextGeneration

```

9. Generate Genetic Algorithm

```

def geneticAlgorithm(population, popSize, eliteSize, mutationRate, generations):
    pop = initialPopulation(popSize, population)
    print("Initial distance: " + str(1 / rankRoutes(pop)[0][1]))

    for i in range(0, generations):
        pop = nextGeneration(pop, eliteSize, mutationRate)

    print("Final distance: " + str(1 / rankRoutes(pop)[0][1]))
    bestRouteIndex = rankRoutes(pop)[0][0]
    bestRoute = pop[bestRouteIndex]
    return bestRoute

```

10. Running the Full Process:

```

bestRoute=geneticAlgorithm(population=cityList, popSize=200, eliteSize=20, mutationRate=0.01, generations=500)

```

Optimum Value generated from GA:

- **Atlanta: 204,972 meters**
- **Philadelphia: 138,632 meters**
- **Cincinnati: 28,622 meters**

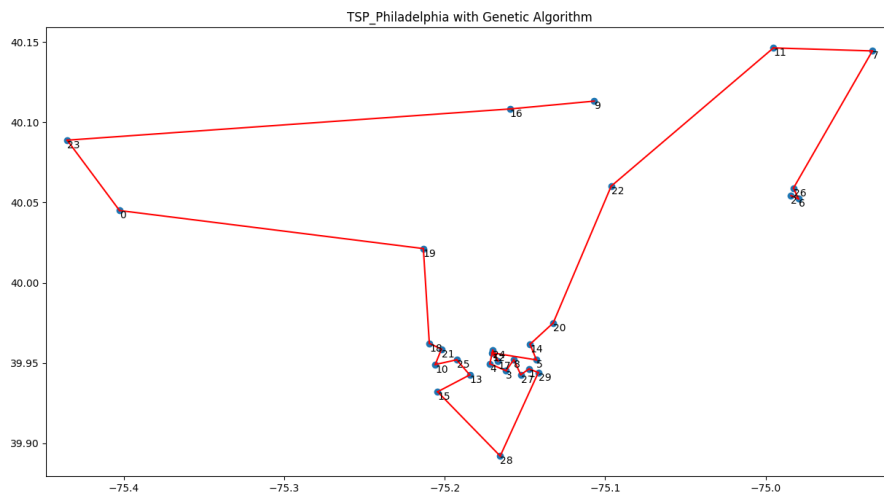
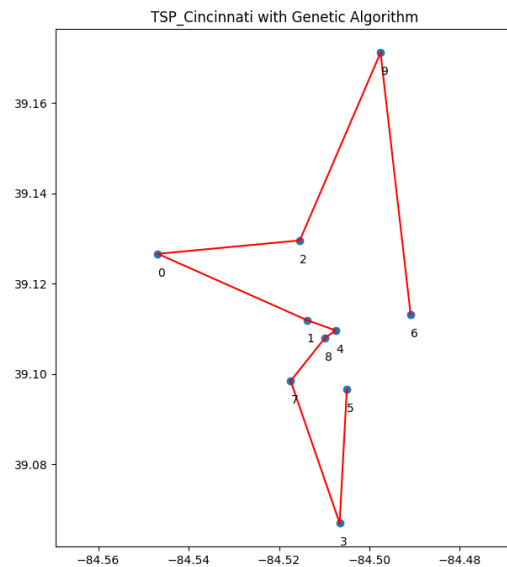
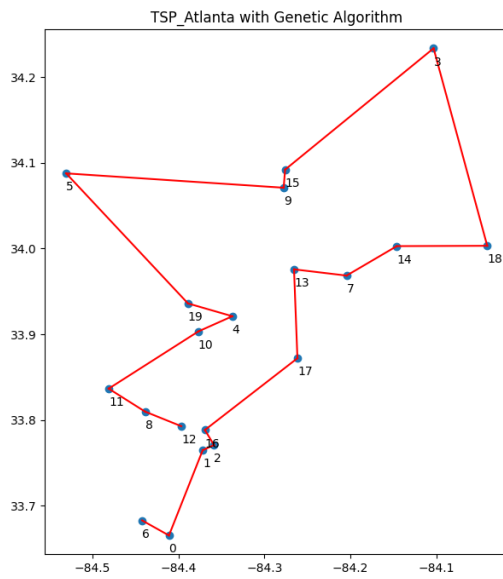
The validation city Atlanta has given the distance of 204,972 meters, which is about 25,000 meter away from the true optimum of 180,000meters. From this we could see the limitation of TSP as a NP problem. However we did see an improvement from the greedy algorithm, which gave us the importance and more applicability of a Genetic Algorithm.

```

# Plot the cities and optimal path
plt.figure(figsize=(7, 8))
plt.scatter(atlanta['Longitude (deg)'], atlanta['Latitude (deg)'])
for i in range(len(atlanta)):
    plt.text(atlanta.loc[i, 'Longitude (deg)'], atlanta.loc[i, 'Latitude (deg)']-0.02, str(atlanta.loc[i, 'Index']))
for i in range(len(city)-1):
    start = atlanta.loc[city[i], ['Longitude (deg)', 'Latitude (deg)']]
    end = atlanta.loc[city[i+1], ['Longitude (deg)', 'Latitude (deg)']]
    plt.plot([start[0], end[0]], [start[1], end[1]], color='red')

plt.title('TSP_Atlanta with Genetic Algorithm')
plt.axis('equal')
plt.show()

```



Solution generated from the greedy algorithm looks different from the solution generated by the genetic algorithm :

The main reason why the solution generated from the greedy algorithm looks different from the solution generated by the genetic algorithm is that the two algorithms use different methods to search for the optimal solution.

The greedy algorithm is a local search algorithm that iteratively improves it by making locally optimal choices. At each step, the algorithm chooses the city that is closest to the current city and has not been visited yet.

On the other hand, the genetic algorithm starts with a population of random solutions and evolves the population over generations to find the optimal solution. The algorithm uses genetic operators such as mutation and crossover to create new solutions from the existing ones. The fitness of each solution is evaluated, and the best solutions are selected for the next generation.

The difference in the search methods used by the two algorithms can lead to differences in the solutions generated. The greedy algorithm may get stuck in a locally optimal solution and fail to find the globally optimal solution. In contrast, the genetic algorithm has the ability to explore a larger search space and find better solutions by combining features from multiple solutions.

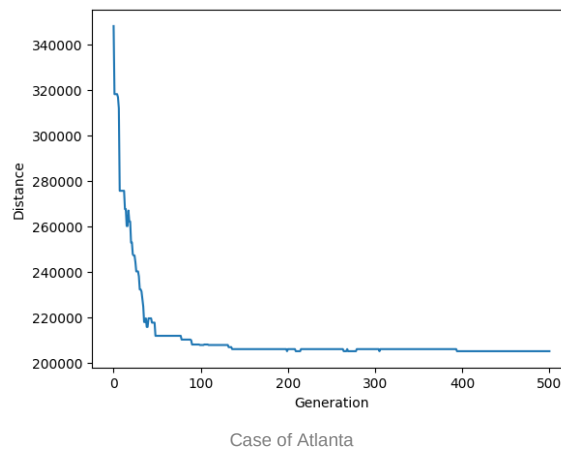
Plot of convergence history is a method of showing the process of how the optimum value is generated by the genetic algorithm as the number of iterations increases.

```
def geneticAlgorithmPlot(population, popSize, eliteSize, mutationRate, generations):
    pop = initialPopulation(popSize, population)
    progress = []
    progress.append(1 / rankRoutes(pop)[0][1])

    for i in range(0, generations):
        pop = nextGeneration(pop, eliteSize, mutationRate)
        progress.append(1 / rankRoutes(pop)[0][1])

    plt.plot(progress)
    plt.ylabel('Distance')
    plt.xlabel('Generation')
    plt.show()

geneticAlgorithmPlot(population=cityList, popSize=200, eliteSize=20, mutationRate=0.01, generations=500)
```



Discussion

- **There are several options of considering if the solution from the genetic algorithm is a global optimum or not.**

In general, genetic algorithms are heuristic methods that provide a good approximation to the global optimum. However, in this case of being provided that a near-optimal solution for the Atlanta city is approximately 180,000 meters, we could determine if the solution obtained from a genetic algorithm is a global optimum. From this validation we could apply the same methodology for other cities. Moreover by using different optimization methodology we could compare the results. However if the validation case is not provided, another method for considering if the solution is global optimum or not is setting the minimum of distance change from the previous to the current and keeping track of changes.

- **The usage Genetic algorithm will not always get the same solution (e.g., the total travel distance is always identical) executing the algorithm multiple times.**

It is not guaranteed that the genetic algorithm will always produce the same solution every time it is executed, even if the same parameters and initial conditions are used. This is because the genetic algorithm involves a random component, such as the initial population and the selection of parents for mating. These random factors can lead to different results each time the algorithm is run, even if the fitness function and other parameters are held constant.

- **From the perspective of designing the algorithm, ways of making more computationally efficient algorithms:**

There could be several ways of making more computationally efficient algorithms. Specifically in the case of Genetic Algorithm the generation of the initial population is an important factor. Therefore we discussed that the generation a greedy algorithm to find the initial population can help to speed up the overall performance of the GA. By combining two algorithms, we could implement a more efficient nearest neighbor algorithm that can find the closest unvisited city more quickly.

Another approach to improve computational efficiency is to implement parallel processing. This can be achieved by breaking the population into smaller groups and running them in parallel, which can reduce the time needed to complete each generation.

Lastly, it is possible that the initial population size is too large or too small for the TSP problem at hand, which can slow down the algorithm. By implementing dynamic population sizing, the algorithm can automatically adjust the size of the population based on the characteristics of the problem, which can improve computational efficiency.

Conclusion

Through this project, our team gained hands-on experience in implementing the genetic algorithm to solve two specific tasks:

- 1) Optimizing a continuous function using the algorithm
- 2) Solving the Traveling Salesman Problem (TSP) using the algorithm

With the first task, we were able to develop a deep understanding of the concept of the genetic algorithm by writing the code step by step without applying open-source libraries. This allowed us to personalize the implementation of the algorithm and comprehend every procedure involved in the process.

Furthermore, by applying the genetic algorithm to solve TSP, we were able to practice using the method to solve real-world problems. This helped us appreciate the algorithm's effectiveness and its potential to tackle complex optimization problems.

Overall, we learned that the genetic algorithm is a powerful method for optimization, and its effectiveness heavily relies on the quality of the initial population, the selection of the right fitness function, and the choice of crossover and mutation operators. Finally, we also learned that there are various ways to improve the efficiency of the algorithm, such as incorporating parallelization techniques or applying hybrid optimization methods.

Works Cited in note citation

ChatGPT, personal communication, April 01, 2023

Gad, Ahmed Fawzy. "Pygad Module." *Pygad Module - PyGAD 2.19.2 Documentation*, https://pygad.readthedocs.io/en/latest/README_pygad_ReadTheDocs.html.

Lewis, Joseph Georgr. "Adapting Genetic Algorithm Packages for the Travelling Salesman Problem." *Medium*, Geek Culture, 15 Sept. 2022, <https://medium.com/geekculture/adapting-genetic-algorithm-packages-for-the-travelling-salesman-problem-ecbdf6c9b99c>.

Prof. Junghyun Kim, [Module_2-1-4 Genetic algorithm - TSP, April 04, 2023]School of Applied Artificial Intelligence, Handong Global University

Prof. Junghyun Kim, [Module_2-1-2,3 Genetic algorithm, March 28, 2023]School of Applied Artificial Intelligence, Handong Global University

"Pygad Module." *Pygad Module - PyGAD 3.0.0 Documentation*, https://pygad.readthedocs.io/en/latest/README_pygad_ReadTheDocs.html#adaptive-mutation.

Reguant, Roc. "How to Implement a Traveling Salesman Problem Genetic Algorithm in Python." *Medium*, Level Up Coding, 12 Jan. 2022, <https://levelup.gitconnected.com/how-to-implement-a-traveling-salesman-problem-genetic-algorithm-in-python-ea32c7bef20f>.

Rocreguant, rocreguant. "Personal_blog/traveling_salesman_problem.ipynb at Main · Rocreguant/personal_blog." *GitHub*, 1 Feb. 2021, https://github.com/rocreguant/personal_blog/blob/main/Genetic_Algorithm_Python_Example/Traveling_Salesman_Problem.ipynb

sTricky. "Python 라이브러리 Haversine 사용법." *The DataBase That i Am Good At*, TISTORY, 29 May 2020, <https://stricky.tistory.com/284>.

Stoltz, Eric. "Evolution of a Salesman: A Complete Genetic Algorithm Tutorial for Python." *Medium*, Towards Data Science, 18 Mar. 2021, <https://towardsdatascience.com/evolution-of-a-salesman-a-complete-genetic-algorithm-tutorial-for-python-6fe5d2b3ca35>.